

FTP - Technical Guidance

Overview [🔗](#)

The goal of this design is to leverage AWS Transfer's FTPS and SFTP support as a means of providing a highly reliable, managed file transfer offering that can be easily and rapidly used across many different Powerschool teams in order to meet their client-facing file transfer needs. This service would support both inbound (customers sending data to PS products), as well as outbound (PS products sending files to customers) file flows. Integrating products can then choose to make use of either flow or both, depending on their needs.

Assumptions [🔗](#)

- The administrative MFE will live in PowerSchool Admin Center, not each product
- Usernames must be globally unique (it is the only identifiable information supplied during an FTP login)
- Files uploaded to an inbound folder will be removed from the server from internet access near instantly and only accessible via internal means (e.g. when a customer uploads a file, if they then try to read it they will likely find it does not exist)

Known Limitations [🔗](#)

- Phase one of PS Admin Center does not allow for setting individual permissions on users for either actions or product scope. As a result, current phase 1 plan is anyone with PS Admin Center access will be the administrator for FTP across their whole district
- Districts will be identified via District Settings UUIDs which means consuming products will need to have this data available as all EB events will reference by District ID
- Unencrypted FTP will not be supported
- No PS employees will have direct access via FTP credentials, files should be obtained, if needed, via secure channels such as limited access in S3 via AWS CLI/console

Requirements [🔗](#)

- Must support file exchange via both FTPS and SFTP protocols. Plain FTP will **not** be supported.
- Customer access must support username/password OR public key based authentication with a strong preference towards pub/priv keys
- Network level protections should be enabled via AWS WAF and AWS Transfer's VPC Endpoint functionality
- IP allow lists should be support so that customers can optionally choose to restrict user-level access on a per-IP basis
- Each customer should be able to provision FTP credentials across multiple folders/directories
 - These should be granular controls, so that different users can have restricted access to only specific file locations within their tenant FTP account
- Internal Powerschool employee access will be granted only via S3
- Users are managed directly by clients in real-time (e.g. saving a new user instantly grants FTP access, deleting a user instantly revokes)
- Customers will self-manage any credentials and PowerSchool will have no way to view the password
- Folders will be declared as either inbound or outbound when created
- Ability to control read/write access on a per user basis
- All actions must be audit logged in accordance with NFR 1022
- Usernames must be globally unique (as there is no other identifying data in the FTP protocol)
- Administrative MFE will live in new PowerSchool Admin Center
- Geo-blocking must be supported by the platform (via AWS WAF & AWS Transfer's VPC endpoint configuration)

- The solution must enable either manual or programmatic disabling of client user credentials that have been inactive beyond an expiration period (supported in Auth0 via either manual action or Auth0 actions)
- **PowerSchool apps will be able to access the files either directly over S3 or over an internally firewalled SFTP access point secured using SSH over only internal networks, S3/EventBridge integration is not mandatory.**

Integrating With the Service [↗](#)

Prerequisites [↗](#)

1. Customers must have a District Settings Universal District ID as this will be used to identify buckets and/or folders to associate to the correct customer
2. Your product must be registered as an [Application](#) in District Settings so that we can properly listen for webhooks for when it is enabled/disabled

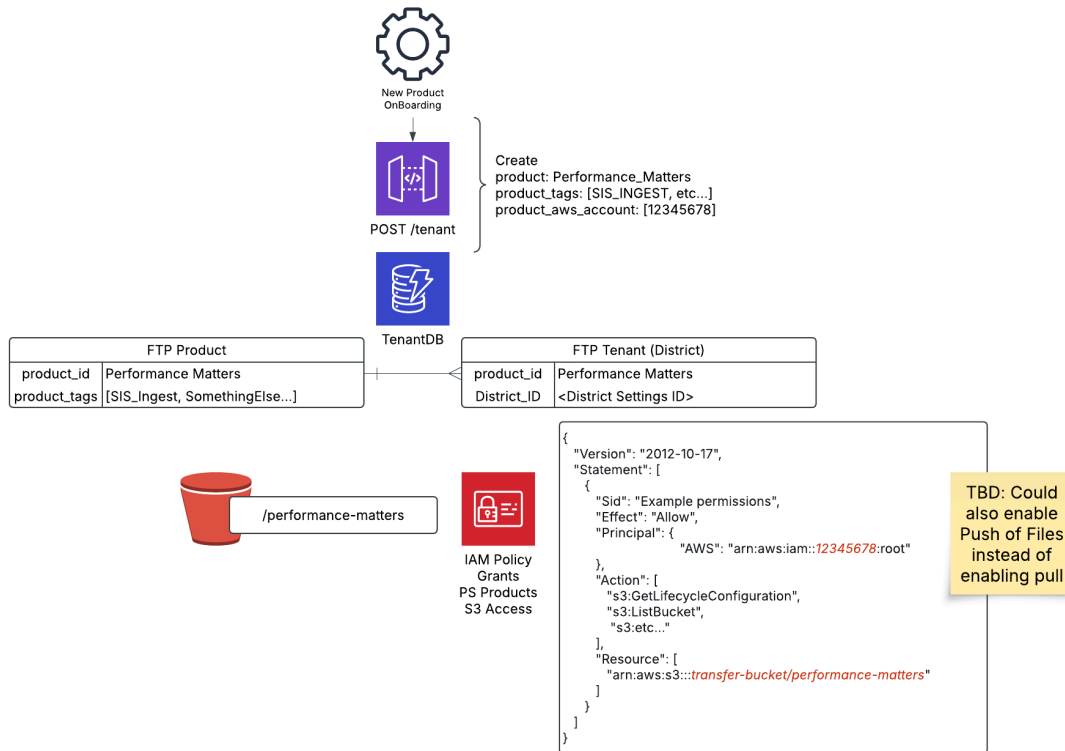
Onboarding a New Product & Tenant [↗](#)

Step 1: Onboarding A New Product Into the Platform [↗](#)

In this scenario, we can imagine a Powerschool Product such as `Performance Matters` is ready to begin using our AWS Transfer Platform for FTPS/SFTP functionality. But before anything can be exposed for their customers, their own product needs the following information:

- Performance Matters needs to understand which SFTP paths are being configured for which purposes in their system. This should be captured as a finite list of **USES** which are maintained in our Tenant system (see JSON payload below for an example).
- For **Inbound** file uploads, Performance Matters needs to determine what functionality they need to receive inbound file uploads (both options can be used together, but most products will likely choose one or the other)
 - Option 1: Performance Matters will connect via their own SFTP credential and periodically poll for new files uploaded by customers.
 - This option is preferred for compatibility with existing implementations or for products that do not have AWS integrations.
 - Option 2: Performance Matters will integrate with AWS Event Bridge to receive notifications of file uploads and process uploaded files by directly accessing them on the associated AWS S3 Buckets
 - This option is preferred for products that already have a strong AWS presence and who are already using similar AWS patterns to manage their inbound file workloads today.
- For **Outbound** file exports, Performance Matters needs to make a similar decision on how they plan to deliver files generated by their system to the FTP location expected by their customers
 - Option 1: Performance Matters connects to the SFTP via the SFTP protocol and pushes the file where it is expected in the FTP Directory
 - Similar to inbound scenarios, this option is preferred when it maintains compatibility with existing application implementations.
 - Option 2: Performance Matters will push the exported file directly to an S3 bucket exposed to them by the AWS Transfer platform.
 - Similar to the inbound option, this option is preferred when the product already has existing AWS experience and where this represents the highest velocity way to achieve export functionality
- If Option 1 is required for either Inbound or Outbound scenarios, Performance Matters needs an FTP username/key to gain access to the internal FTP server
- If Option 2 is required for either Inbound or Outbound scenarios, Performance Matters needs access to the underlying S3 bucket.
- If necessary, this same API can also support the provisioning of customer subscriptions via district-id.

The above needs are achieved by invoking a Tenant API with a repeatable, secure CREATE-TENANT operation. This enables the platform to store relevant product metadata about their FTP featureset for later use, and also allows our platform to authorize future credential requests on behalf of the customers of this Powerschool Product. This API also (optionally) allows products to request an IAM permission be granted so that they can access their FTP buckets from one or more AWS accounts under their control. More about that use case is described in the following sections. This API call would be secured using an Auth0 M2M token.



Pseudo-API Call [🔗](#)

POST /product

```
1 {
2   "productCode" : "PM",
3   "productUses": ["Scan", "QTI", "Enrollment"],
4   "productPublicKey": "keycontentshere",
5   "IAMPolicy": "stringified_json_of_IAM_policy"
6 }
```

Step 2: Onboarding a customer [🔗](#)

The FTP Service will be listed in District Settings as a product. When enabled, this would fire a webhook that informs the FTP service that the customer has the FTP product, as well as the other products that customer has. The service would then also receive webhooks whenever other products are added/removed allowing the FTP Tenant records to be automatically updated.

Use Cases [🔗](#)

Use Case 1: Customer Configures an SFTP Credential To Use for Importing Data Into Powerschool (“InBound” Use Case) And PS Product Wants to Access The File Via S3 [🔗](#)

Most products support inbound SFTP use cases where a customer who wants to set up an SFTP user for the purpose of uploading files into a Powerschool Product. These files, once uploaded over SFTP, are usually further processed by the PS application. This means that, in most scenarios, the PS Product will require a notification and some metadata to indicate to it that a file has been uploaded for a specific purpose.

Host Application

Home > Transfer Admin MFE > Manage Users > New Folder

Username

Type

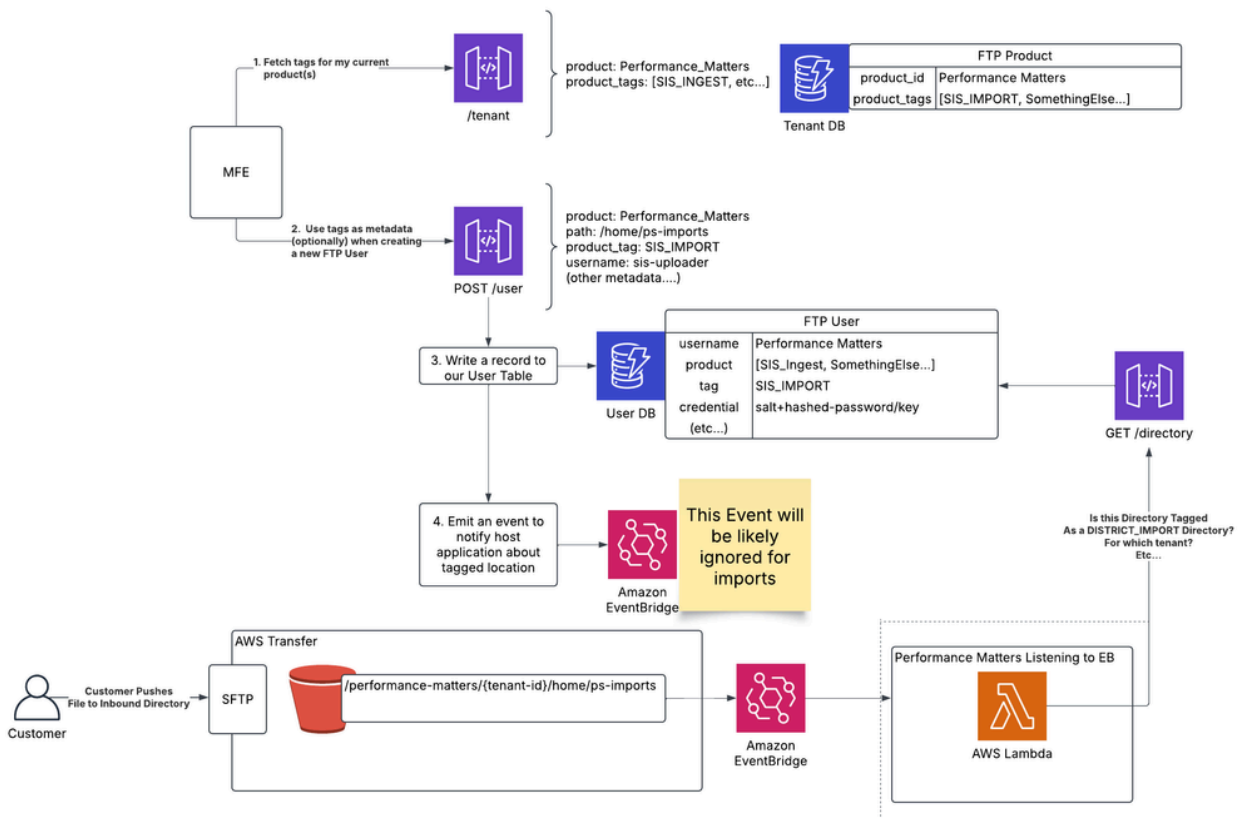
Folder

Access

IP Whitelists

While some PS products look for these import files by polling the FTP location using an FTP credential, other products with AWS-native architectures rely on accessing AWS infrastructure directly to access these files. In order to support the second integrating use case, the platform needs to support notifications that are emitted *when a file is added to our S3 bucket inside AWS Transfer*. PS Applications that rely on inbound files to be accessed via AWS can respond to these events and pull files from the AWS Transfer S3 bucket.

Note that inbound files would be promptly swept off the S3 bucket connected to the SFTP to a separate S3 bucket that can be accessed by PowerSchool products but is inaccessible to the outside world.



Use Case 2: Customer Configures an SFTP Credential To Use for Exporting Data Out Of Powerschool (“Outbound” Use Case) And PS Product Wants to Access The File Via S3

After onboarding, a customer should also be able to configure in our MFE an SFTP credential that can be used for pulling data out of a PS Product. This step needs to meet two criteria:

- The customer must be able to self-service an SFTP credential with a specific file directory they can use exclusively for the intended export purpose
- The PS Product that is responsible for placing files on this location needs to be informed if/when the customer creates it, and for what purpose it was created, so that they can programmatically place the file the customer expects to get in the correct location.
 - Our sense is that most products do this today as part of their manual SFTP credential exchange, but a self-service model will likely require programmatic triggers that the host application can respond to instead.

The below mockups show an extremely simplified version of the UX a customer might experience when setting this up.

Host Application

Home > Transfer Admin MFE > Manage Folders > New Folder

Product

Performance Matters

Use

QTI

Directory

text

Access

Inbound

Save

Host Application

Home > Transfer Admin MFE > Manage Users

New User

Username	Type	Folder	Access	IPs	Edit
user1	SSH Key	/pm/enroll	Read	0.0.0.0/0	☐
user2	Password	/pm/scan	Write	1.2.3.4/32	☐
user3	Password	/sgy/usage	ReadWrite	1.2.3.4/32	☐

Host Application

Home > Transfer Admin MFE > Manage Users > New Folder

Username

user4

Type

SSH Key

Generate

Folder

/pm/qti

Access

Write

IP Whitelists

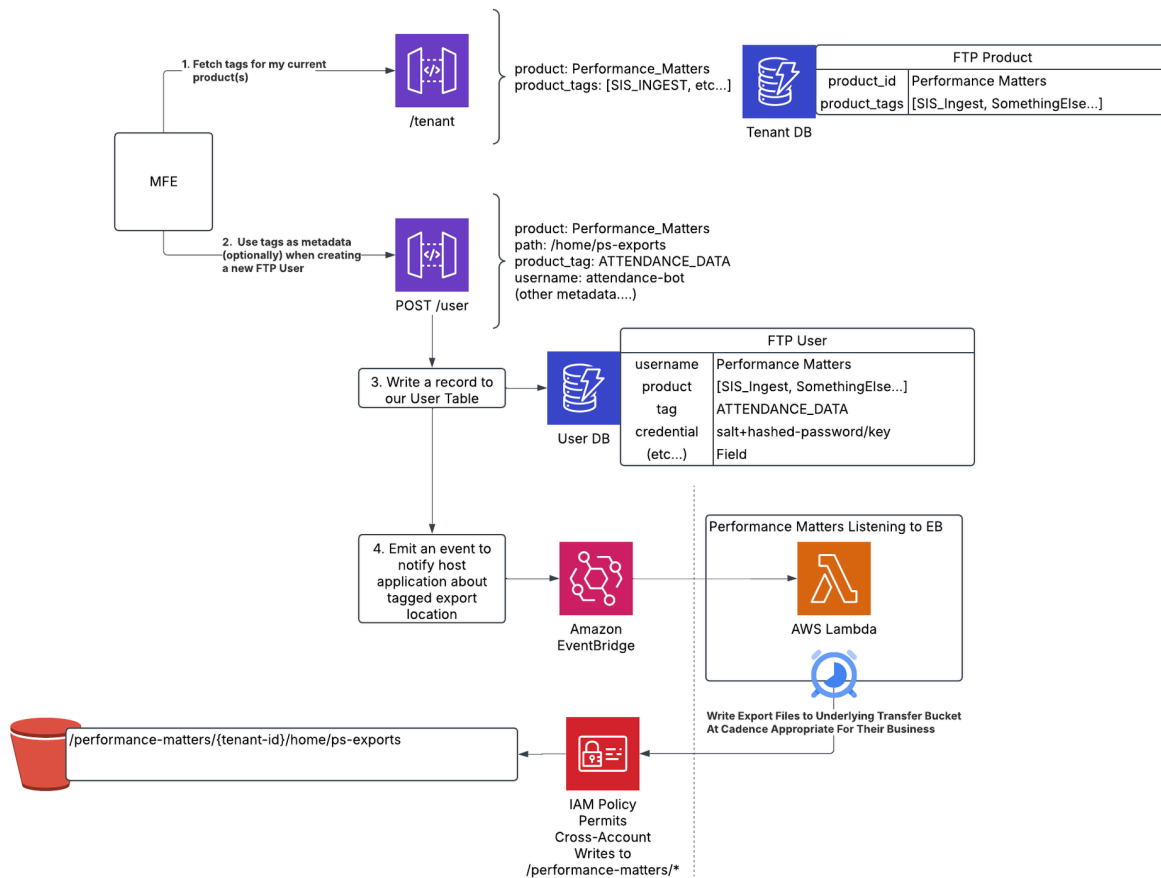
0.0.0.0/0

Save

Of note on this screen is the following information:

- User Name - behind the scenes this must be globally unique. We may need to generate uuid suffixes for all usernames at the service layer.
- Folder - Our proposal here is that each user has their own “home directory” where they access files, and we do not support, at least initially, users with multiple directories.
- Use: The Tag field represents one of the `productUses` that was provided to our platform service when the Performance Matters App was initialized in our Tenant API. This tag is OPTIONAL but using it here is how the customer can indicate to our system that this SFTP location is where they expect the application to export, in this example, attendance data.

Behind the scenes, this is what happens in our system when the above user provisioning action is taken:



Not pictured in this diagram: Authorization Logic

We will likely consider a Lifecycle on how long files sit on outbound SFTP but this is TBD based on product input.

Use Case 3: Customer Configures an SFTP Credential To Use for Importing Data Out Into Powerschool (“Inbound” Use Case) And PS Product Wants to Access The File Via SFTP

Products would mostly access this SFTP via the same means they do today. When the customer uploads the file, it will be moved to an S3 bucket that supports FTP access via an Internal AWS Transfer Service. Your product would connect over SFTP much as it does today with a few tight security restrictions. The product will have read/write access to this bucket which allows for intermediate “working directory” activities as identified in use cases such as SchoolMessenger.

- The SFTP will be locked down heavily via networking/VPC restrictions so it is not internet accessible
- Access will only be allowed via SSH and you must use the credentials passed during onboarding [FTP - Technical Guidance | Pseud](#) [API Call](#) to access the product-specific bucket. Within this bucket files will be arranged within folders for each District Settings District ID that has an FTP with the files in that folder then mirroring the ones configured by the user.

Use Case 4: Customer Configures an SFTP Credential To Use for Exporting Data Out Of Powerschool (“Outbound” Use Case) And PS Product Wants to Access The File Via S3

Similar to the inbound use case, the product will have access to the S3 bucket via SFTP. The product would authenticate via the same RSA key and would upload the file to the appropriate directory via SFTP. This would again be heavily network restricted. Customers would then be able to access this file.

We will likely consider a Lifecycle on how long files sit on outbound SFTP but this is TBD based on product input.

Migration Frequently Asked Questions

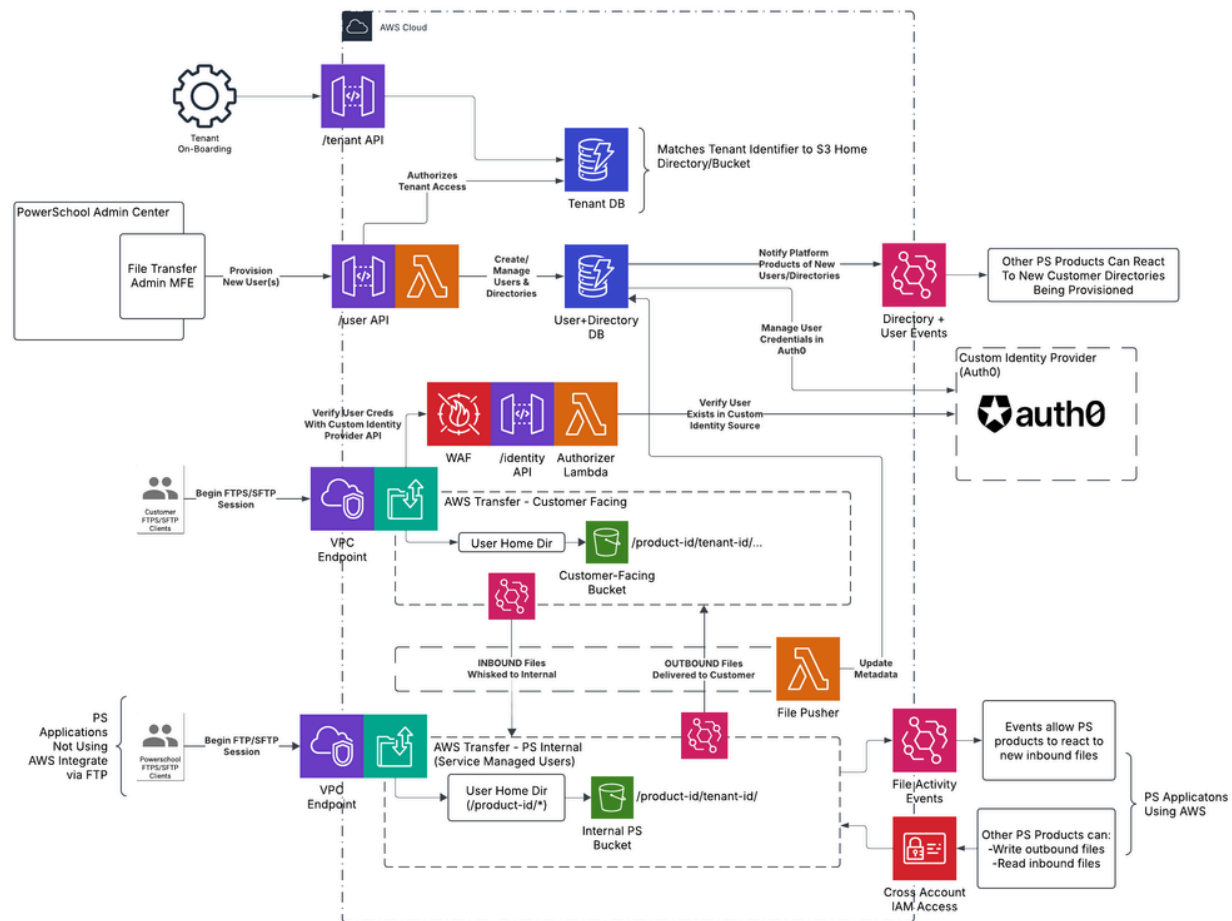
- How do we drive adoption?
 - We need to extoll the value of this service. Customers self-managing means they never wait on PowerSchool for FTP credentials. When they have an employee leave the district and they want to rotate the password, they can do this immediately on their own. When they have a new software solution they've purchased that they wish to integrate with PowerSchool, again, no need to work with support. They can do this on their own. This provides faster turn around time for requests and puts them in the driver seat. This should be pitched as an awesome additional system PowerSchool is offering, not a burden. We can also talk about ease of maintenance and enhanced security. Now FTP will be managed in a central way and with greater security including user specific accounts managed by the distrit and RSA key based authentication even for products that didn't previously support it,
- Is this self serve?
 - All accounts would be 100% self-managed by the customers with PowerSchool having no access to credentials
- Can we support both FTPs at once?
 - You will almost certainly have to. Since the new service will be self-managed, it will be impossible to migrate all customers on the same time. Even for a particular customer, it's very likely there will be some use cases that can migrate quicker than others. E.g. they may be able to change their SIS to drop files in a new SFTP rapidly. But updating data warehouse that pulls in various 3rd party assessment reports might be managed by a different team with longer lead time. As a result you should plan on supporting both for a reasonable time. The product teams should consider a situation where the files dropped on either server end up in a single location and processing occurs from there. This allows the product to support a single code path and both FTPs simultaneously.
- How will I know if a customer has switched over so I can shut off their old FTP?
 - Since you receive an EventBridge event when a folder is setup, you know when a customer is intending to use the new FTP. You will also receive an EventBridge event as soon as they drop a file indicating usage. However, we won't have a clear way to know the old SFTP (legacy FTP) has been abandoned via this new service. Splunk logs will allow you to identify when a username hasn't been used for a sufficiently long time to deactivate the account.
- Can we automatically set everything up for customers?
 - No, and this is intentional. We do not have their current credentials and even for situations where we may have their current credential, it defeats the point of ensuring the passwords are rotated and only know to the end user as part of this effort.
- I need an FTP in Canada or another region for contractual obligations, what should I do?
 - Other than the fact that the URLs would differ and the S3 buckets would differ, this should not impact your estimation effort. We can spin up additional instances in AWS regions supporting Transfer Family based on business need.
- My product isn't onboarded into PowerSchool Admin Center, what do I need to do?
 - Admin center doesn't actually exist yet, so no product is yet integrated. That said, the act of integrating should not involve work for your team. The Admin Center will provide net new functionality for customers to self-manage FTP credentials. There is one exception here - since the AIS team does already have self-managed FTP (our research indicates they are the only product with this functionality), AIS will have work to reeducate their customers to use this new workflow if they implement this new FTP service instead of their current self-managed solution.

Technical Design

Our design leverages the following components:

Component	Implementation Notes
AWS Transfer Service	Provides FTPS and SFTP functionality, mapped onto S3 bucket(s) with distinct S3 Paths for each tenant identifier. Configured to use a custom identity provider which we will configure using our own AWS API Gateway (see next component for details). This API GW integration is documented by AWS here Support for FTPS also requires use of a VPC endpoint, since FTPS is not supported for public endpoints.

	To better support requirements for internal access and rapid transfer of inbound files away from customer-facing SFTP directories, we will stand up two AWS Transfer servers. • A public server will contain SFTP credentials managed by our customers • An internal PS Server will receive copies of files uploaded by our customers and will enable either FTP or AWS based access to those files for internal PS employees/applications.
"Identity API" Gateway to Custom Identity Provider (WAF + API GW + Lambda)	To support username/password login, in addition to key-based login, we need to provide a custom identity provider for AWS Transfer to use when initializing sessions. Furthermore, our requirement to integrate WAF and IP allow lists provide additional justification to use an API Gateway here in lieu of a simple Lambda function. This /identity API is responsible for receiving user credentials and validating them, using an Authorizer Lambda, against our chosen identity store (Auth0 at the time of this writing).
"User API"	This API is used by our File Transfer MFE and is built on our standard reference architecture. Its API endpoint are what allow customers to self-provision their own SFTP credentials. When new credentials are provisioned, this API is responsible for appropriately updating the system's internal database as well as external dependencies (AWS Transfer + Auth0).
File Transfer Admin MFE	This will be the MFE UI that connects to our User API and enables dynamic, just-in-time user management. Our current requirements are scoped such that this MFE is only planned for integration with the Powerschool Admin Center.
"Tenant API"	This API is invoked in order to bootstrap and configure new products using the Platform. It configures and stores information identifying the product's home directories and defined USES for FTP directories (these USES are later used by the MFE to determine the intended purpose of an FTP location). Lastly, this Tenant API also dynamically provisions IAM policies, when applicable, for products that require underlying access to S3 Buckets containing FTP files.
Event. Bridge Notifications	Any PS Product that runs on AWS today may optionally choose to listen for Event Bridge notifications from this platform service, focused on two scenarios: • Products can receive an event whenever a customer chooses to create or retire a new SFTP Directory with a product-defined USE. ◦ This event is intended for use by products that need to respond to this event by exporting data files to customer-determined locations. • Products can receive an event whenever a new file is uploaded by the customer and moved to the "PS Internal" FTP S3 bucket.



Identity Provider Selection Criteria [🔗](#)

Service Managed Users: Supports only SSH key so does this not meet our requirement of supporting username/password based logins as well. ❌

Secrets Manager: Least scalable (and more costly for large user counts). Also concerns over direct storage of user credentials. But has been successfully used by other teams and may be considered sufficient for initial/small use cases. In discussing with AWS they did not recommend this approach and agreed the cost considerations would not be scalable ❌

Cognito: Scalable, AWS native, but needs to be managed by the team who owns the solution. Due to the management aspect of this solution, as well as cost, we decided not to go with this approach.

Custom DynamoDB: Create a custom DynamoDB that stores the usernames, SSH public keys, and/or salted-hashed passwords in a custom database. This would be highly scalable and easily extensible but we are then rolling our own security which comes with some inherent risks.

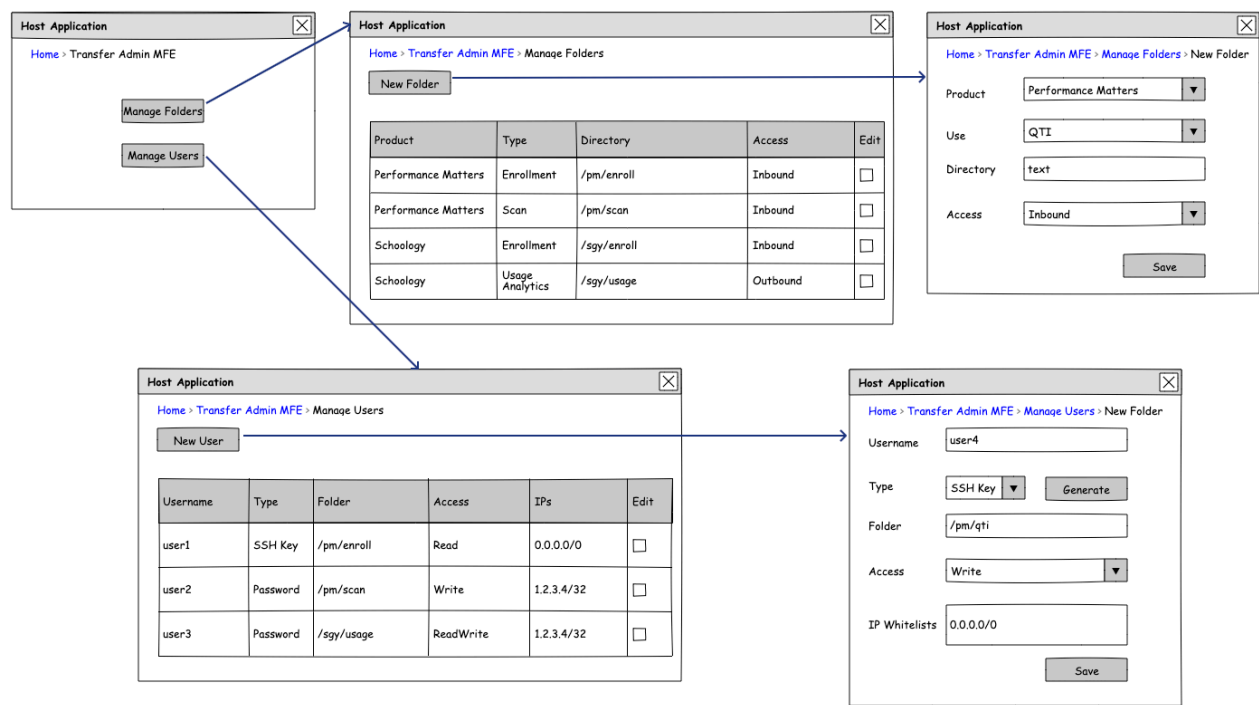
Auth0: Likely most scalable and centrally managed, but some concerns over use case.

Auth0 approach [🔗](#)

We would setup our own Auth0 tenant for this solution (`ps-transfer`) which would have both prod and pre-prod tenants. Within that tenant we can create a new database for FTP user credentials. All aspects of the tenant would disable things like MFA, email, etc. as these are not possible to use with an FTP based solution. We would however have the possibility of taking advantage of Auth0 protections such as IP restrictions (since we are sending this from a central IP we can lock down any attempt at brute-force of the Resource Owner Password flow token API), and we can also enable brute force protection against the various accounts. For more details on the Auth0 approach, see Appendix B.

MFE Mockups [🔗](#)

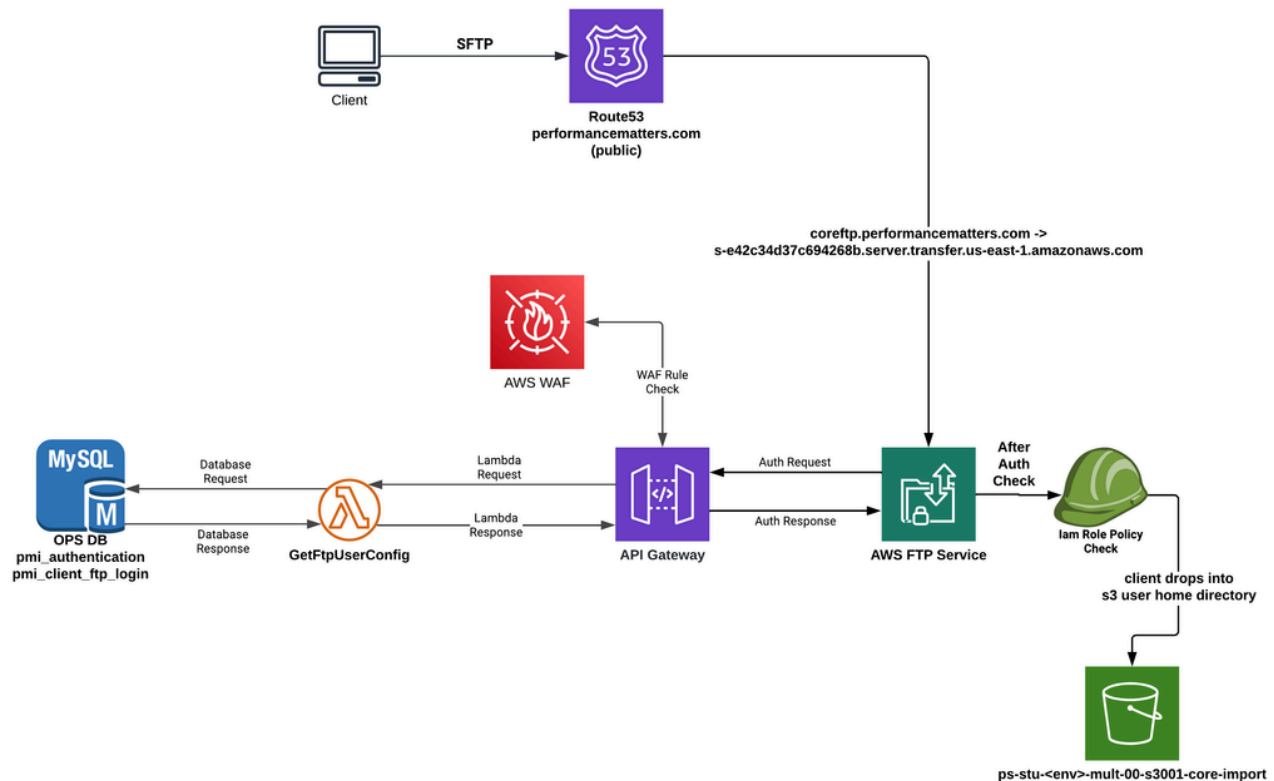
The following shows the UI that a district administrator will use to configure both the folder locations and FTP users. Note the MFE would be housed im PowerSchool Admin Center and is NOT something consuming products need to scope.



Appendix A - Note on Existing Powerschool Implementations [🔗](#)

Multiple teams in Powerschool have already leveraged AWS Transfer to meet their FTP needs. Though not all requirements listed here were necessary for these teams, there is prior experience successfully delivering an FTP experience to customers using AWS Transfer with the custom identity provider approach.

- AIS Design Document: [📄 Use AWS Transfer service for sending files via SFTP protocol](#)
- Performance Matters Arch Diagram



- AWS Sample Project with Transfer's FTP + Username/Password: <https://github.com/aws-samples/ftp-with-password-authentication-cdk-sample>

Appendix B - Auth0 Notes [🔗](#)

To use Auth0 as the user account store we would implement the “Resource Owner Password flow”. With that flow (which is really just an API call and response similar to the “Client Credentials flow”), our authentication lambda would make a call to the Auth0 Authentication API with (1) the client id+secret for our lambda (this is created once as an Application in Auth0), (2) the username and password sent by the user and forwarded to our lambda in the context, and (3) the IP address of the end user/system, which is also forwarded to our lambda in the context. The response from Auth0 would indicate success (in which case we get the information about that “user”, which could in turn contain the information we need to set up for FTP access including the home directory and any permission settings) or failure (with error codes indicating a general username+password not found error or a brute force / rate limiting error).

The main concern with using this approach is that it takes advantage of an OAuth 2.0 flow (Resource Owner Password, or ROP) which is generally advised against in almost all cases. In fact, using this flow is counter to the OAuth 2.0 Security Best Practices document, and as such is likely to be removed from the OAuth 2.1 spec when/if that comes out. The main issue with this flow is that it is unique in the OAuth arsenal in allowing the client (in this case, our FTP authorizer) to “handle” the user credentials (specifically, the user’s password); removing it from the 2.1 standard then means that using OAuth guarantees that the resource provider never has to see the user’s credentials. Advice I have found indicates that they would prefer the direct use of LDAP or a similar credentials-validation service in the case where the caller unavoidably has access to the user-entered id and password (as is our situation).

Before signing on to using Auth0 here, we should **discuss this with our Auth0 consultants** to see if they have any qualms about this usage or knowledge about this flow being deprecated inside of Auth0. The Auth0 documentation does indicate this should be a “last resort” option, but it is fully documented and appears to be fully supported.

Additional Resources (for the team if we go down this path):

- [Avoid Common Issues with Resource Owner Password Flow and Attack Protection](#)
- [Is it disallowed or only discouraged to use Resource Owner flow and what is the alternative?](#)

F Awaiting confirmation SSH keys are supported, if yes we will likely move forward with Auth0.

Open Technical Questions [🔗](#)

- Does Auth0 support RSA Keys?
 - We have a backup plan if it does not which does not impact uptake but a small increase in scope of building the service itself
- How do PS Apps access data in the customer drop points?
 - Does each app need an RSA key protected login that has access to all of that products folders?
 - Does each product need a specific password set up for each customer?